# Programming Assignment 1

EC5504 Computational Photography, Fall 2026
Prof. Jae Woong Soh

Due Friday, Oct. 23, 2026, at 11:59 PM KST

## 1. Camera RAW Processing and Image Signal Processing

In this assignment, you will implement a simplified camera image signal processing (ISP) pipeline. Unlike conventional RGB images such as JPEG or PNG, a RAW image contains measurements obtained directly from an image sensor. Therefore, several processing steps are required before the RAW measurements can be displayed as a natural color image.

You will start from a RAW image stored in the DNG format. You may either capture your own DNG image using a camera or use the DNG file provided with this assignment. You will use `rawpy` only to read the RAW image and extract necessary camera metadata. Unless otherwise specified, all subsequent image processing steps must be implemented by yourself using Python and `NumPy`.

## Part 1. Reading RAW Sensor Data

A color image sensor does not directly measure an RGB vector at every pixel. Instead, each sensor element measures only one color component according to a color filter array (CFA), typically a Bayer pattern.

Use `rawpy` to read the provided DNG file.

```
import rawpy
import numpy as np

with rawpy.imread('input.dng') as raw:
    raw_image = raw.raw_image_visible.copy()

    print(raw_image.shape)
    print(raw_image.dtype)
    print(raw.raw_pattern)
    print(raw.color_desc)
    print(raw.black_level_per_channel)
    print(raw.white_level)
    print(raw.camera_whitebalance)
```

Report and briefly explain the following information:

- Image width and height
- Data type and bit depth
- Bayer pattern
- Black level
- White level
- Camera white-balance gains

Visualize the RAW sensor image in grayscale. For all visualizations, you must provide **zoomed-in** views to enable detailed analysis and comparison of the results.

**Question:** Why does the RAW image contain only one intensity value at each pixel instead of three RGB values?

# Part 2. Linearization

The RAW sensor measurement does not necessarily have zero intensity when no light reaches the sensor. Due to sensor offset and dark current, a non-zero *black level* exists. In addition, sensor measurements above the *white level* correspond to saturated pixels.

Convert the RAW image into a normalized linear image within the range $[0, 1]$. Do this by applying a linear transformation (shift and scale) to the image, so that the *black level* is mapped to 0, and the *white level* is mapped to 1. Then, clip values outside the valid range, *i.e.,* negative values to 0, and values greater than 1 to 1.

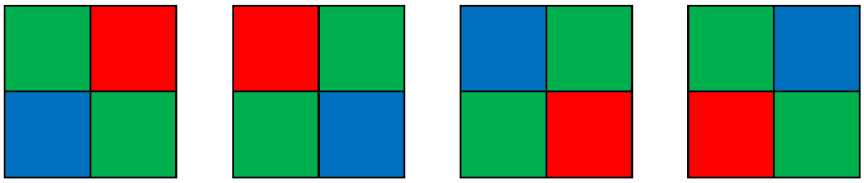
If different color channels have different black levels, apply the corresponding black level according to the Bayer pattern.

Visualize the linearized RAW image and describe what you have observed.

**Question:** Why should black-level correction be performed before subsequent ISP operations? What is the effective bit depth of the RAW image?

# Part 3. Bayer Pattern

Most digital cameras use a Bayer color filter array.



Four possible $2 \times 2$ Bayer patterns from left to right: `'grbg'`, `'rggb'`, `'bggr'`, `'gbrg'`.

## 3.1. Determining the Bayer Pattern from the Measurements

Determine the Bayer pattern of your RAW image by analyzing the sensor measurements and intermediate processing results. Do not use the Bayer pattern information provided by `rawpy` or other RAW-processing libraries.

Consider the four possible Bayer patterns (GRBG, RGGB, BGGR, and GBRG), and identify the pattern that produces the most physically plausible result. Explain how you determined the correct pattern.

After identifying the pattern, construct the corresponding red, green, and blue sampling masks $M_R$, $M_G$, $M_B$, and visualize the CFA sampling structure.

## 3.2 Comparison with the Bayer Pattern from the Metadata

Obtain the Bayer pattern from the RAW metadata and compare it with the pattern identified in Part 3.1.

Visualize the Bayer pattern using these masks. Including a small crop of the RAW image together with the corresponding CFA pattern is recommended.

Explain how an incorrect Bayer pattern would affect the final RGB image.

# Part 4. White Balance

The spectral sensitivities of the red, green, and blue sensor channels are different. Furthermore, the illumination affects the measured RGB values. White balancing compensates for these differences by multiplying each color channel by a corresponding gain:

$$\begin{bmatrix} I'_R \\ I'_G \\ I'_B \end{bmatrix} = \begin{bmatrix} g_R & 0 & 0 \\ 0 & g_G & 0 \\ 0 & 0 & g_B \end{bmatrix} \begin{bmatrix} I_R \\ I_G \\ I_B \end{bmatrix}.$$

Implement the following three white-balancing methods.

## 4.1. Gray-World White Balance

The gray-world assumption assumes that the average color of a scene is achromatic:

$$\mu_R \approx \mu_G \approx \mu_B.$$

Compute appropriate red and blue gains with respect to the green channel and apply them directly to the Bayer RAW image.

## 4.2. White-World White Balance

The white-world assumption assumes that the brightest response in each color channel corresponds to a white surface.

Determine the gain of each color channel using its maximum intensity.

## 4.3. Camera White Balance

Use the camera white balance gains stored in the RAW metadata.

Normalize the gains such that the green gain is equal to 1.

After completing the entire ISP pipeline, compare the final images obtained using the three white-balancing methods.

**Question:** Which white-balancing method produces the most natural result? Explain why the results differ.

# Part 5. Demosaicing

At this point, each sensor location still contains only one color component. Demosaicing estimates the two missing color components at every pixel to generate a full-resolution RGB image.

For example, for an RGGB sensor, the red measurements are sparsely sampled as

$$\begin{matrix} R & \cdot & R & \cdot & R \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ R & \cdot & R & \cdot & R \\ \cdot & \cdot & \cdot & \cdot & \cdot \end{matrix}$$

Implement **bilinear demosaicing** to interpolate missing red, green, and blue values.

```
def demosaic_bilinear(raw, pattern):
    ...
    return rgb
```

where the output has dimensions

$$H \times W \times 3$$

with channel order $[R, G, B]$.

You must implement bilinear interpolation from scratch, without using built-in interpolation functions. You may **not** use existing demosaicing functions provided by `rawpy`, OpenCV, or other image-processing libraries.

Visualize the demosaiced RGB image. Discuss the color characteristics.

**Question:** Find a region containing edges or fine textures and describe artifacts produced by bilinear demosaicing.

## Part 6. Camera RGB to Linear sRGB

The demosaiced RGB image is represented in a camera-specific RGB color space. Since the camera RGB values are determined by the spectral sensitivities of the image sensor, they do not directly correspond to the standard sRGB color space.

A camera color space can be related to linear sRGB through the XYZ color space. Conceptually, the transformation from linear sRGB to camera RGB can be written as

$$\mathbf{c}_{cam} = M_{\mathrm{XYZ}\to cam} M_{\mathrm{sRGB}\to XYZ} \mathbf{c}_{sRGB},$$

where $M_{\mathrm{sRGB}\to XYZ}$ converts linear sRGB values to the device-independent XYZ color space, and $M_{\mathrm{XYZ}\to cam}$ maps XYZ values to the camera-specific RGB color space.

The standard linear sRGB-to-XYZ matrix is

$$M_{\mathrm{sRGB}\to XYZ} = \begin{bmatrix} 0.4124564 & 0.3575761 & 0.1804375 \\ 0.2126729 & 0.7151522 & 0.0721750 \\ 0.0193339 & 0.1191920 & 0.9503041 \end{bmatrix}.$$

Therefore, the inverse mapping transforms camera RGB into linear sRGB:

$$\mathbf{c}_{sRGB} = \left(M_{\mathrm{XYZ}\to cam} M_{\mathrm{sRGB}\to XYZ}\right)^{-1} \mathbf{c}_{cam}.$$

In practice, `rawpy` provides a camera color transformation matrix through `rawpy.color_matrix`. Denote the corresponding camera-to-linear-sRGB transformation as $M_{\mathrm{cam}\to\mathrm{sRGB}}$.

Use this matrix to transform the demosaiced camera RGB values into linear sRGB.

Visualize and compare the image before and after color-space correction.

**Question:** Why does the demosaiced camera RGB image have incorrect colors even after white balancing?

## Part 7. Brightness Adjustment

The resulting image is still a *linear* RGB image. Its intensity range may not be appropriate for display.

Adjust image brightness by multiplying all RGB values by a common scale factor:

$$I_{\mathrm{bright}} = \alpha I_{\mathrm{linear}}.$$

Select $\alpha$ such that the mean grayscale intensity becomes a target value $m$ (you may use `rgb2gray`):

$$\alpha = \frac{m}{\mathrm{mean}(I_{\mathrm{gray}})}.$$

Experiment with several target values, for example

$$m = 0.15, \quad 0.20, \quad 0.25, \quad 0.30.$$

After brightness adjustment, clip values above 1.

Report the target value that produces the most visually pleasing result.

## Part 8. sRGB Gamma Encoding

Even after brightness adjustment, the image is still represented in linear RGB. A linear RGB image generally appears too dark when directly displayed.

Convert the linear RGB values to nonlinear sRGB values using

$$C_{\text{sRGB}} = \begin{cases} 12.92 C_{\text{linear}}, & C_{\text{linear}} \leq 0.0031308, \\ 1.055 C_{\text{linear}}^{1/2.4} - 0.055, & C_{\text{linear}} > 0.0031308, \end{cases}$$

where $C \in \{R, G, B\}$.

Visualize and compare the linear RGB with the gamma-encoded sRGB image.

**Question:** Why does a linear RGB image appear dark when it is directly displayed on a standard monitor?

## Part 9. Complete Your Camera ISP Pipeline

Combine all previous steps into your own camera ISP pipeline. Implement a function similar to

```
def camera_isp(raw_image, metadata):

    x = linearize(raw_image, metadata)

    x = white_balance(x, metadata)

    x = demosaic_bilinear(x, metadata)

    x = color_correction(x, metadata)

    x = adjust_brightness(x)

    x = srgb_encode(x)

    return x
```

Include representative intermediate results from each major stage in your report.

## Part 10. Comparison with RAW Processor and Camera JPEG

Finally, compare the image produced by your ISP pipeline with (1) an image generated using `rawpy`'s built-in processing pipeline, and (2) the JPEG image produced by the camera.

You may use `rawpy.postprocess()` **only in this section**.

For example,

```
with rawpy.imread('input.dng') as raw:
    reference = raw.postprocess(
        use_camera_wb=True,
        output_bps=16
    )
```

Refer to the documentation for details on these parameters. [1]

Compare the following three images: your implementation, the rawpy post-processing result, and the camera-generated JPEG image. The camera JPEG provided with the assignment was generated by the manufacturer's full in-camera ISP pipeline.

Visualize the three results side by side and discuss their differences, e.g., color, brightness, contrast, sharpness, and overall appearance.

**Question:** Which of the three results do you prefer, and what processing differences do you think are responsible for their different appearances?

**Important:** `rawpy.postprocess()` must not be used for implementing Parts 1–9.

## 2. Image Filtering

In this problem, you will implement fundamental image filtering techniques, and learn how images are processed through filtering. Furthermore, you will understand how filtering can enhance or extract features from images.

## Part 1. Basic Filtering with 2D Convolution

Implement 2D convolution manually. Compare runtime with built-in functions (e.g., `scipy.signal.convolve2d`). Make sure to handle image boundaries appropriately using proper padding. Test with a simple averaging filter (box filter).

## Part 2. Gaussian Blur

- Implement a function to generate a 2D Gaussian kernel given the standard deviation $\sigma$:

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

- Convolve the Gaussian kernel with an image.
- Visualize and compare the results for different values of $\sigma$.

## Part 3. Edge Detection

(a) **Sobel Filter**

Implement Sobel filters in $x$ and $y$ directions.

$$S_X = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}, \quad S_Y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

[1] `https://letmaik.github.io/rawpy/api/rawpy.Params.html`

Then, compute gradient magnitude.

$$M(x, y) = \sqrt{I_x^2 + I_y^2}$$

(b) **Laplacian of Gaussian (LoG)**

- Implement Laplacian of Gaussian kernel:

$$LoG(x, y) = \nabla^2 G(x, y) = \frac{\partial^2 G}{\partial x^2} + \frac{\partial^2 G}{\partial y^2}$$

- Detect edges by identifying zero-crossings in the LoG response.
- Compare the edge maps obtained using Sobel and LoG. Discuss the differences between first-order and second-order edge detection.

## Submission Requirements

- Source code
- Output and intermediate results
- Report including:
  - Explanation of your implementation
  - Output and intermediate results
  - Analyses and discussions
  - Answers to all questions

## Academic Integrity and Use of AI Tools

You are expected to understand and be able to explain all submitted code. The core algorithms required in this assignment must be implemented by you.
If generative AI tools are used for auxiliary purposes, their use must be clearly disclosed in the report. Generative AI must not be used to generate complete assignment solutions.